

1. RNN의 두 가지 근본 문제

(1) 병목(bottleneck): 이전 문맥이 고정 크기 벡터 하나에 압축됨 → 긴 시퀀스 정보 손실.

(2) 순차 계산: 시점 순서대로만 계산 → 병렬화 불가.

해결: (1)→Attention, (2)→Transformer(Self-Attention). Transformer는 (1),(2) 둘 다 해결.

2. RNN + Attention (문제(1) 해결)

동기: 생성되는 각 토큰은 서로 다른 source 토큰과 관련됨.

① 점수(score) — 디코더 상태 s_t 와 인코더 은닉 h_i 의 유사도(dot product):

$$\text{score}(s_t, h_i) = s_t^T h_i$$

② 분포(distribution):

$$e^t = [s_t^T h_1, \dots, s_t^T h_N]$$

$$\alpha^t = \text{softmax}(e^t)$$

③ 문맥벡터(attention value):

$$a_t = \sum_{i=1}^N \alpha_i^t h_i$$

④ 결합 후 예측:

$$\xi_t = \tanh(W_c[a_t; s_t] + b_c)$$

$$\hat{y}_t = \text{softmax}(W_y \xi_t + b_y)$$

3. Self-Attention 개념

각 원소가 입력의 모든 원소에 attend하여 자기 표현을 정제(= 모든 원소의 가중합).

가정: 입력을 서로 유기적으로 관련된 여러 원소로 분할 가능(문장의 단어, 영상의 프레임 등). → 병목(1) 해결.

4. Q · K · V (★ 핵심)

- **Query:** 정보를 모으려는 원소
- **Key:** attend 대상 원소들의 특성
- **Value:** key 원소가 담은 실제 정보

입력 x 에 FC weight 곱: $Q=xW^Q, K=xW^K, V=xW^V$

절차: ① QK^T 로 점수 → ② $\sqrt{d_k}$ 로 나눠 스케일 →

③ softmax(=분포) → ④ $\Sigma(\text{분포} \times V)$ =출력.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$d_k=Q, K$ 의 차원(예시 64). $\sqrt{d_k}$ 스케일링: dot product가 커져 softmax가 포화/gradient 소실되는 것 방지. → 행렬연산으로 병렬화 → 문제(2) 해결.

5. Encoder (입력 문맥 이해)

Multi-Head Attention

여러 head로 서로 다른 표현 부분공간(subspace)·위치의 정보에 동시에 attend → 표현력 ↑.

$$d_{\text{model}} = d_v \times \text{num_heads}, \quad d_v = d_{\text{model}} / \text{num_heads}$$

각 head 출력 concat → W^O 곱 → 최종 출력.

Add & Norm

Add(잔차연결):

$$H(x) = x + \text{MultiHead}(x)$$

Layer Norm: 각 샘플의 feature를 정규화. (γ, β 학습 파라미터)

$$\hat{x}_{i,k} = \frac{x_{i,k} - \mu_i}{\sqrt{(\sigma_i^2 + \epsilon)}}$$

$$\ln_i = \gamma \cdot \hat{x}_i + \beta = \text{LayerNorm}(x_i)$$

★ **Layer Norm은 학습·추론 절차가 동일**(배치 통계에 의존 X). Batch Norm과 대비.

정규화	정규화 축(평균/분산 계산 범위)
Batch	배치 N + 공간 H,W (채널 C별)
Layer	채널 C + 공간 H,W (샘플 N별)
Instance	공간 H,W만 (채널·샘플별)

※ feature가 벡터면 (H,W)=1.

Feed Forward Network (FFN)

$$\text{FFN}(x) = \text{Act}(xW_1 + b_1)W_2 + b_2$$

FC → 활성화함수(예: ReLU) → FC = **2-layer MLP**.

Stacked Blocks (N ×)

self-attention 블록을 **N개 적층**. 두 번째부터는 이전 블록의 출력 임베딩을 입력으로 받음. 마지막 층은 **입력과 같은 길이**의 변환된 토큰 시퀀스 출력.

Positional Encoding

Transformer는 입력을 순차로 받지 않음 → **위치 정보를 임베딩에 더해** 주임.

$$\text{PE}_{(\text{pos}, 2i)} = \sin(\text{pos} / 10000^{2i/d_{\text{model}}})$$

$$\text{PE}_{(\text{pos}, 2i+1)} = \cos(\text{pos} / 10000^{2i/d_{\text{model}}})$$

sinusoid를 쓰는 이유: ①위치마다 고유 ②값이 [-1,1]로 유계 ③상대 위치 인코딩 가능.

6. 학습 vs 추론 (★ 자주 출제)

학습(Teacher Forcing): 정답(GT) 시퀀스를 디코더 입력으로 한 번에 투입 → 순차처리 불필요, **병렬 계산** 가능. 그래서 **masked attention** 필요.

추론: 디코딩은 단어 하나씩 순차적으로 예측.

7. Decoder (출력 생성)

① Masked Multi-Head Attention

학습 시 입력을 한 번에 받으므로 단어 예측할 때 **미래 단어를 참조**해버림 → 차단 필요.

방법: attention score 행렬의 **미래 위치에 $-\infty$ 대입** → softmax 후 **0** → 참조 차단(causal mask).

② Encoder-Decoder Attention

$$K, V \leftarrow \text{인코더} / Q \leftarrow \text{디코더}$$

디코더 Query가 인코더가 만든 문맥(K,V)을 조회.

③ Prediction

Linear(FC): 디코더 출력 → class score(출력 크기 = **vocab size**) → softmax. **[EOS]** 나올 때까지 반복.

8. 장점 / 단점

장점

- **긴 시퀀스 강함:** self-attention이 원거리 토큰 관계를 직접 모델링
- **높은 병렬화:** 토큰 동시 처리 → GPU 학습 빠름
- **입력 유연성:** 순서 없는 집합·순서 시퀀스 모두 처리

단점

- **비용 큼:** attention이 시퀀스 길이에 **$O(n^2)$** (메모리·계산)
- **데이터 형그리:** 효과적 학습에 대량 데이터 필요

빠른 암기 체크

• 핵심식: **$\text{softmax}(QK^T/\sqrt{d_k})V$**

• Q=정보 요청, K=특성, V=내용

• $\sqrt{d_k}$ = softmax 포화 방지

• Multi-head: **concat**→ W^O , $d_v=d_{\text{model}}/h$

• 잔차: $H=x+\text{MHA}(x)$, **LayerNorm(학습=추론)**

• FFN = FC→ReLU→FC

• PE: 짝수 sin / 홀수 cos

• Mask: 미래에 $-\infty$ → softmax 0

• Enc-Dec attn: **$K,V=\text{enc} / Q=\text{dec}$**

• Teacher Forcing→병렬→mask 필요